DEPARTMENT OF COMPUTER STUDIES

(Applications Development and Emerging Technologies)

PRE-SUMMATIVE ASSESSMENT

2

PHP OPERATORS AND CONTROL STRUCTURE

Student Name / Group Name:	Castillo, Jade Carlos A.
Members (if Group):	**Name** / **Role** ( ) / ( ) ( ) / ( ) ( ) / ( )
Section:	TW22
Professor:	Joseph Calleja

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements.

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Understand and apply best practices and standards in the development of website.

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- To understand the different types of operators that are available on PHP.
- To know what is operator precedence and operator associativity in PHP.
- To use escape sequence properly in the program.
- To know the different approach of control structures.
- To know the fundamentals syntax for conditional and looping structures.
- To properly use the compound expression using the logical operators.
- To know the rules of break, continue, and goto statements.

IV. BACKGROUND INFORMATION

PHP Operators and Control Structures

Arithmetic Operator

Example	Label	Outcome
$a + $b	Addition	Sum of $a and $b
$a - $b	Subtraction	Difference of $a and $b
$a * $b	Multiplication	Product of $a and $b
$a / $b	Division	Quotient of $a and $b
$a % $b	Modulus	Remainder of $a divided by $b

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.124

Assignment Operator

Example	Label	Outcome
$a = 5	Assignment	$a equals 5
$a += 5	Addition-assignment	$a equals $a plus 5
$a *= 5	Multiplication-assignment	$a equals $a multiplied by 5
$a /= 5	Division-assignment	$a equals $a divided by 5
$a .= 5	Concatenation-assignment	$a equals $a concatenated with 5

Beginning PHP and MySQL 3rd Edition by: W. Jason Gilmore pp.126

Try following example to understand all the arithmetic operators.

```
<html>
   <head>
      <title>Arithmetical Operators</title>
   </head>
   <body>
      <?php
         $a = 42;
         $b = 20;
         $c = $a + $b;
         echo "Addition Operation Result: $c <br/>";
         $c = $a - $b;
         echo "Subtraction Operation Result: $c <br/>";
         $c = $a * $b;
         echo "Multiplication Operation Result: $c <br/>";
         $c = $a / $b;
         echo "Division Operation Result: $c <br/>";
         $c = $a % $b;
         echo "Modulus Operation Result: $c <br/>";
         $c = $a++;
         echo "Increment Operation Result: $c <br/>";
         $c = $a--;
         echo "Decrement Operation Result: $c <br/>";
      ?>
   </body>
</html>
```

This will produce the following result –

```
Addition Operation Result: 62
Subtraction Operation Result: 22
Multiplication Operation Result: 840
Division Operation Result: 2.1
Modulus Operation Result: 2
Increment Operation Result: 42
Decrement Operation Result: 43
```

PHP Conditional Statements

Very often when you write code, you want to perform different actions for different conditions. You can use conditional statements in your code to do this.

In PHP we have the following conditional statements:

- `if` statement - executes some code if one condition is true
- `if...else` statement - executes some code if a condition is true and another code if that condition is false
- `if...elseif...else` statement - executes different codes for more than two conditions
- `switch` statement - selects one of many blocks of code to be executed

PHP - The if Statement

The `if` statement executes some code if one condition is true.

Syntax

```
if (condition) {
  code to be executed if condition is true;
}
```

Example

```
<?php
$t = date("H");

if ($t < "20") {
  echo "Have a good day!";
}
?>
```

Output "Have a good day!" if the current time (HOUR) is less than 20:

PHP – The if...else Statement

The `if...else` statement executes some code if a condition is true and another code if that condition is false.

Syntax

```
if (condition) {
  code to be executed if condition is true;
} else {
  code to be executed if condition is false;
}
```

Example

```
<?php
$t = date("H");
if ($t < "20") {
  echo "Have a good day!";
} else {
  echo "Have a good night!";
}
?>
```

Output "Have a good day!" if the current time is less than 20, and "Have a good night!" otherwise:

PHP - The if...elseif...else Statement

The `if...elseif...else` statement executes different codes for more than two conditions.

Syntax

```
if (condition) {
  code to be executed if this condition is true;
} elseif (condition) {
  code to be executed if first condition is false and this condition is
true;
} else {
  code to be executed if all conditions are false;
}
```

Example

Output "Have a good morning!" if the current time is less than 10, and "Have a good day!" if the current time is less than 20. Otherwise it will output "Have a good night!":

```
<?php
$t = date("H");

if ($t < "10") {
  echo "Have a good morning!";
} elseif ($t < "20") {
  echo "Have a good day!";
} else {
```

```
  echo "Have a good night!";
}
?>
```

The PHP switch Statement

Use the `switch` statement to **select one of many blocks of code to be executed**.

Syntax

```
switch (n) {
  case label1:
    code to be executed if n=label1;
    break;
  case label2:
    code to be executed if n=label2;
    break;
  case label3:
    code to be executed if n=label3;
    break;
    ...
  default:
    code to be executed if n is different from all labels;
}
```

Example

```
<?php
$favcolor = "red";

switch ($favcolor) {
  case "red":
    echo "Your favorite color is red!";
    break;
  case "blue":
    echo "Your favorite color is blue!";
    break;
  case "green":
    echo "Your favorite color is green!";
    break;
  default:
    echo "Your favorite color is neither red, blue, nor green!";
}
?>
```

This is how it works: First we have a single expression *n* (most often a variable), that is evaluated once. The value of the expression is then compared with the values for each case in the structure. If there is a match, the block of code associated with that case is executed. Use `break` to prevent the code from running into the next case automatically. The `default` statement is used if no match is found.

PHP Loops

Often when you write code, you want the same block of code to run over and over again a certain number of times. So, instead of adding several almost equal code-lines in a script, we can use loops.

Loops are used to execute the same block of code again and again, as long as a certain condition is true.

In PHP, we have the following loop types:

- `while` - loops through a block of code as long as the specified condition is true
- `do...while` - loops through a block of code once, and then repeats the loop as long as the specified condition is true
- `for` - loops through a block of code a specified number of times
- `foreach` - loops through a block of code for each element in an array

PHP while Loop

The `while` loop - Loops through a block of code as long as the specified condition is true.

The `while` loop executes a block of code as long as the specified condition is true.

Syntax

```
while (condition is true) {
  code to be executed;
}
```

Example

The example below displays the numbers from 1 to 5:

```php
<?php
$x = 1;

while($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
?>
```

Example Explained

- $x = 1; - Initialize the loop counter ($x), and set the start value to 1
- $x <= 5 - Continue the loop as long as $x is less than or equal to 5
- $x++; - Increase the loop counter value by 1 for each iteration

PHP do while Loop

The `do...while` loop - Loops through a block of code once, and then repeats the loop as long as the specified condition is true.

The `do...while` loop will always execute the block of code once, it will then check the condition, and repeat the loop while the specified condition is true.

Syntax

```
do {
  code to be executed;
} while (condition is true);
```

Example

The example below first sets a variable $x to 1 ($x = 1). Then, the do while loop will write some output, and then increment the variable $x with 1. Then the condition is checked (is $x less than, or equal to 5?), and the loop will continue to run as long as $x is less than, or equal to 5:

```php
<?php
$x = 1;
```

```
do {
  echo "The number is: $x <br>";
  $x++;
} while ($x <= 5);
?>
```

Note: In a `do...while` loop the condition is tested AFTER executing the statements within the loop. This means that the `do...while` loop will execute its statements at least once, even if the condition is false. See example below.

PHP for Loop

The `for` loop - Loops through a block of code a specified number of times.

The `for` loop is used when you know in advance how many times the script should run.

Syntax

```
for (init counter; test counter; increment counter) {
  code to be executed for each iteration;
}
```

Parameters:

- *init counter*: Initialize the loop counter value
- *test counter*: Evaluated for each loop iteration. If it evaluates to TRUE, the loop continues. If it evaluates to FALSE, the loop ends.
- *increment counter*: Increases the loop counter value

Example

The example below displays the numbers from 0 to 10:

```
<?php
for ($x = 0; $x <= 10; $x++) {
  echo "The number is: $x <br>";
}
?>
```

PHP foreach Loop

The `foreach` loop - Loops through a block of code for each element in an array.

The `foreach` loop works only on arrays, and is used to loop through each key/value pair in an array.

Syntax

```
foreach ($array as $value) {
  code to be executed;
}
```

For every loop iteration, the value of the current array element is assigned to $value and the array pointer is moved by one, until it reaches the last array element.

Example

The example will output the values of the given array ($colors):

```
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value) {
  echo "$value <br>";
}
?>
```

PHP Break and Continue

You have already seen the `break` statement used in an earlier chapter of this tutorial. It was used to "jump out" of a `switch` statement.

PHP Break

The `break` statement can also be used to jump out of a loop.
This example jumps out of the loop when **x** is equal to **4**:

Example

```
<?php
for ($x = 0; $x < 10; $x++) {
  if ($x == 4) {
    break;
  }
```

```
  echo "The number is: $x <br>";
}
?>
```

PHP Continue

The `continue` statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of **4**:

Example

```
<?php
for ($x = 0; $x < 10; $x++) {
  if ($x == 4) {
    continue;
  }
  echo "The number is: $x <br>";
}
?>
```

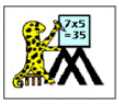
V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

1. Using PHP operators create a length conversion page, integrated with HTML and CSS (note: use formula for each conversion)
   Example: 1 meter = 100 centimeter

   Sample Output:

Name Date



MEASURE CONVERSION CHART – LENGTHS (UK)

METRIC CONVERSIONS					
1 centimetre	=	10 millimetres	1 cm	=	10 mm
1 decimetre	=	10 centimetres	1 dm	=	10 cm
1 metre	=	100 centimetres	1 m	=	100 cm
1 kilometre	=	1000 metres	1 km	=	1000 m

IMPERIAL CONVERSIONS					
1 foot	=	12 inches	1 ft	=	12 in
1 yard	=	3 feet	1 yd	=	3 ft
1 chain	=	22 yards	1 ch	=	22 yd
1 furlong	=	220 yards (or 10 chains)	1 fur	=	220 yd (or 10 ch)
1 mile	=	1760 yards (or 8 furlongs)	1 mi	=	1760 yd (or 8 fur)

METRIC -> IMPERIAL CONVERSIONS					
1 millimetre	=	0.03937 inches	1 mm	=	0.03937 in
1 centimetre	=	0.39370 inches	1 cm	=	0.39370 in
1 metre	=	39.37008 inches	1 m	=	39.37008 in
1 metre	=	3.28084 feet	1 m	=	3.28084 ft
1 metre	=	1.09361 yards	1 m	=	1.09361 yd
1 kilometre	=	1093.6133 yards	1 km	=	1093.6133 yd
1 kilometre	=	0.62137 miles	1 km	=	0.62137 mi

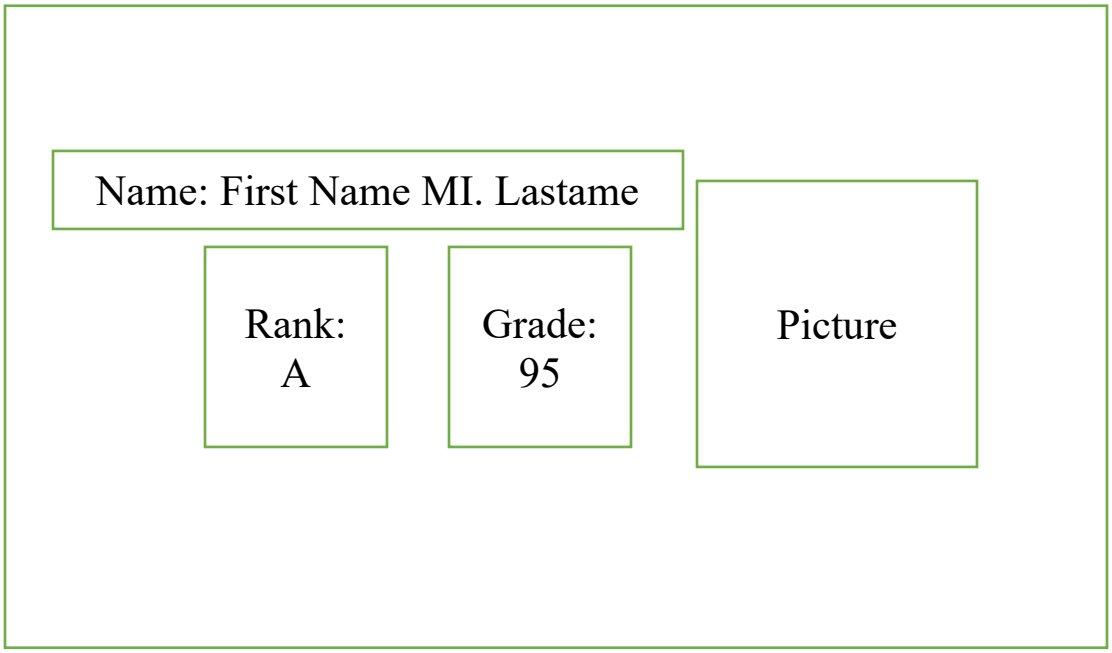
IMPERIAL -> METRIC CONVERSIONS					
1 inch	=	2.54 centimetres	1 in	=	2.54 cm
1 foot	=	30.48 centimetres	1 ft	=	30.48 cm
1 yard	=	91.44 centimetres	1 yd	=	91.44 cm
1 yard	=	0.9144 metres	1 yd	=	0.9144 m
1 mile	=	1609.344 metres	1 mi	=	1609.344 m
1 mile	=	1.609344 kilometres	1 mi	=	1.609344 km

2. Using conditional statement create a grade ranking program, integrated with HTML and CSS.
   Example
   Grade = 92 Ranking: A-

Use the equivalents below

A: 93-100
A-: 90-92
B+: 87-89
B: 83-86
B-: 80-82
C+: 77-79
C: 73-76
C-: 70-72
D+: 67-69
D: 63-66
D-: 60-62
F: Below 60

Sample Output



3. Using Looping Statements write a program which will give you all of the potential combinations of a two-digit decimal combination, printed in a comma delimited format :

   Sample output :
   00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99,

Snip and paste your source codes here. Snip it directly from the IDE so that colors of the codes are preserved for readability. Include additional pages if necessary.

- **Part 1 – Measure Conversion Table**
  - **HTML/PHP**

index.php

3t2526 > module1 > tfa2 > act1 > index.php > ...

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Formative 2 - Act 1 - Measure Conversion Chart</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="page">
        <div class="header">
            <span>Name</span>
            <span class="date">Date</span>
        </div>

        <h1>MEASURE CONVERSION CHART - LENGTHS (UK)</h1>

            <?php
                $cm = 1;
                $dm = 1;
                $m = 1;
                $km = 1;
                $ft = 1;
                $yd = 1;
                $ch = 1;
                $fur = 1;
                $mi = 1;
                $mm = 1;
                $inch = 1;
```

index.php

3t2526 > module1 > tfa2 > act1 > index.php > ...

```php
                $cm_to_mm = $cm * 10;
                $dm_to_cm = $dm * 10;
                $m_to_cm = $m * 100;
                $km_to_m = $km * 1000;

                $ft_to_in = $ft * 12;
                $yd_to_ft = $yd * 3;
                $ch_to_yd = $ch * 22;
                $fur_to_yd = $fur * 220;
                $mi_to_yd = $mi * 1760;

                $mm_to_in = $mm * 0.03937;
                $cm_to_in = $cm * 0.39370;
                $m_to_in = $m * 39.37008;
                $m_to_ft = $m * 3.28084;
                $m_to_yd = $m * 1.09361;
                $km_to_yd = $km * 1093.6133;
                $km_to_mi = $km * 0.62137;

                $in_to_cm = $inch * 2.54;
                $ft_to_cm = $ft * 30.48;
                $yd_to_cm = $yd * 91.44;
                $yd_to_m = $yd * 0.9144;
                $mi_to_m = $mi * 1609.344;
                $mi_to_km = $mi * 1.609344;
            ?>
```

```php
        <table>
        <tr class="title"><th colspan="5">METRIC CONVERSIONS</th></tr>
        <tr><td>1 centimetre</td><td>=</td><td><?php echo $cm_to_mm; ?> millimetres</td><td>1 cm</td><td>= 10 mm</td></tr>
        <tr><td>1 decimetre</td><td>=</td><td><?php echo $dm_to_cm; ?> centimetres</td><td>1 dm</td><td>= 10 cm</td></tr>
        <tr><td>1 metre</td><td>=</td><td><?php echo $m_to_cm; ?> centimetres</td><td>1 m</td><td>= 100 cm</td></tr>
        <tr><td>1 kilometre</td><td>=</td><td><?php echo $km_to_m; ?> metres</td><td>1 km</td><td>= 1000 m</td></tr>
        </table>

        <table>
            <tr class="title"><th colspan="5">IMPERIAL CONVERSIONS</th></tr>
                <tr><td>1 foot</td><td>=</td><td><?php echo $ft_to_in; ?> inches</td><td>1 ft</td><td>= 12 in</td></tr>
                <tr><td>1 yard</td><td>=</td><td><?php echo $yd_to_ft; ?> feet</td><td>1 yd</td><td>= 3 ft</td></tr>
                <tr><td>1 chain</td><td>=</td><td><?php echo $ch_to_yd; ?> yards</td><td>1 ch</td><td>= 22 yd</td></tr>
                <tr><td>1 furlong</td><td>=</td><td><?php echo $fur_to_yd; ?> yards</td><td>1 fur</td><td>= 220 yd</td></tr>
                <tr><td>1 mile</td><td>=</td><td><?php echo $mi_to_yd; ?> yards</td><td>1 mi</td><td>= 1760 yd</td></tr>
        </table>

        <table>
            <tr class="title"><th colspan="5">METRIC -> IMPERIAL CONVERSIONS</th></tr>
                <tr><td>1 millimetre</td><td>=</td><td><?php echo $mm_to_in; ?> inches</td><td>1 mm</td><td>= 0.03937 in</td></tr>
                <tr><td>1 centimetre</td><td>=</td><td><?php echo $cm_to_in; ?> inches</td><td>1 cm</td><td>= 0.39370 in</td></tr>
                <tr><td>1 metre</td><td>=</td><td><?php echo $m_to_in; ?> inches</td><td>1 m</td><td>= 39.37008 in</td></tr>
                <tr><td>1 metre</td><td>=</td><td><?php echo $m_to_ft; ?> feet</td><td>1 m</td><td>= 3.28084 ft</td></tr>
                <tr><td>1 metre</td><td>=</td><td><?php echo $m_to_yd; ?> yards</td><td>1 m</td><td>= 1.09361 yd</td></tr>
                <tr><td>1 kilometre</td><td>=</td><td><?php echo $km_to_yd; ?> yards</td><td>1 km</td><td>= 1093.6133 yd</td></tr>
                <tr><td>1 kilometre</td><td>=</td><td><?php echo $km_to_mi; ?> miles</td><td>1 km</td><td>= 0.62137 mi</td></tr>
        </table>
```

```php
        <table>
            <tr class="title"><th colspan="5">IMPERIAL -> METRIC CONVERSIONS</th></tr>
                <tr><td>1 inch</td><td>=</td><td><?php echo $in_to_cm; ?> centimetres</td><td>1 in</td><td>= 2.54 cm</td></tr>
                <tr><td>1 foot</td><td>=</td><td><?php echo $ft_to_cm; ?> centimetres</td><td>1 ft</td><td>= 30.48 cm</td></tr>
                <tr><td>1 yard</td><td>=</td><td><?php echo $yd_to_cm; ?> centimetres</td><td>1 yd</td><td>= 91.44 cm</td></tr>
                <tr><td>1 yard</td><td>=</td><td><?php echo $yd_to_m; ?> metres</td><td>1 yd</td><td>= 0.9144 m</td></tr>
                <tr><td>1 mile</td><td>=</td><td><?php echo $mi_to_m; ?> metres</td><td>1 mi</td><td>= 1609.344 m</td></tr>
                <tr><td>1 mile</td><td>=</td><td><?php echo $mi_to_km; ?> kilometres</td><td>1 mi</td><td>= 1.609344 km</td></tr>
        </table>
    </div>
</body>
</html>
```

- **CSS**

style.css

3t2526 > module1 > tfa2 > act1 > # style.css > body

```css
body {
    font-family: Calibri, sans-serif;
    background: #ffffff;
}

.page {
    width: 800px;
    margin: 20px auto;
}

.header {
    display: flex;
    justify-content: space-between;
    margin-bottom: 10px;
}

span.date{
    position: relative;
    left: -450px;
}

h1 {
    font-family: Cambria, serif;
    font-weight: normal;
    color: #2554a3;
    font-size: 26px;
    margin-bottom: 15px;
}

```

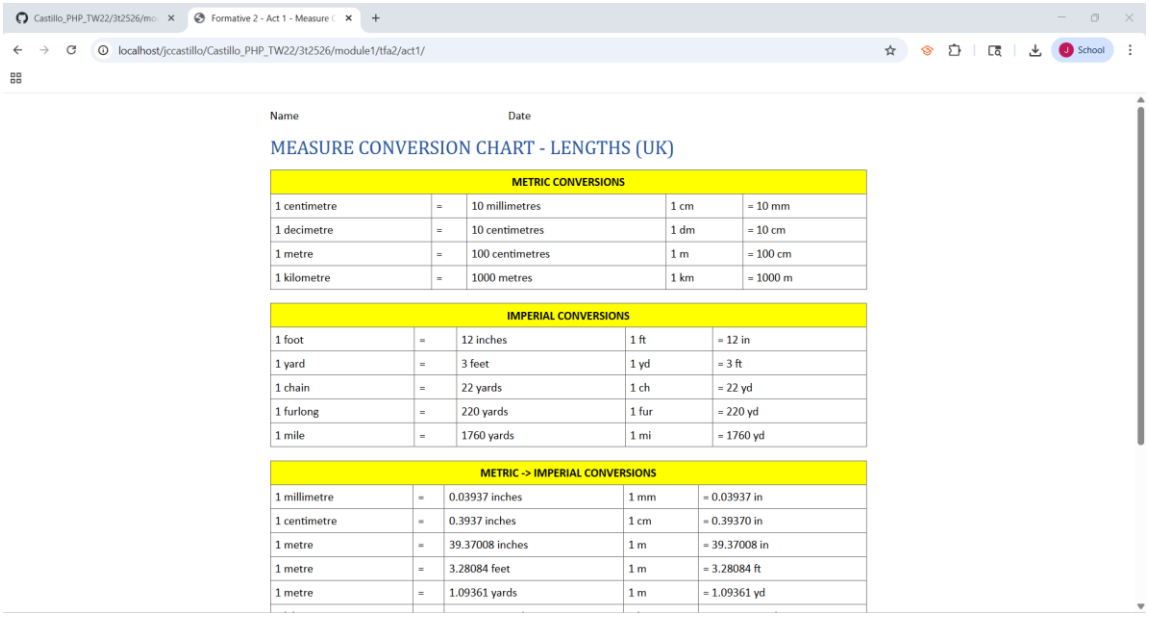
```css
table {
    width: 100%;
    border-collapse: collapse;
    margin-bottom: 18px;
}

td, th {
    border: 1px solid #666;
    padding: 6px;
}

.title th {
    background: yellow;
    text-align: center;
    font-weight: bold;
}

```

- **Website Output**



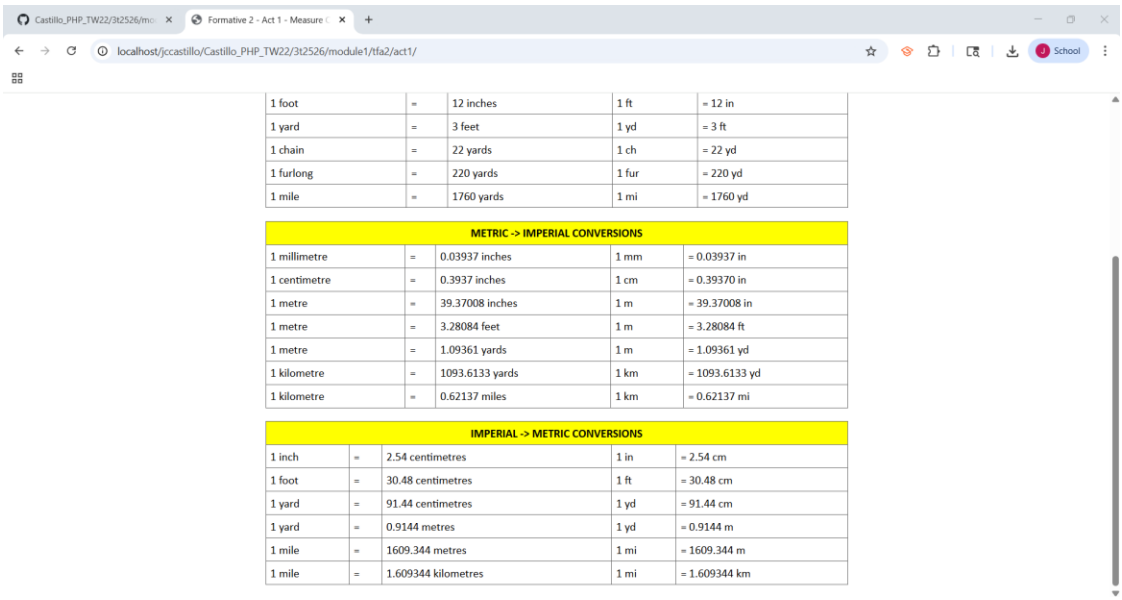
Name Date

MEASURE CONVERSION CHART - LENGTHS (UK)

METRIC CONVERSIONS				
1 centimetre	=	10 millimetres	1 cm	= 10 mm
1 decimetre	=	10 centimetres	1 dm	= 10 cm
1 metre	=	100 centimetres	1 m	= 100 cm
1 kilometre	=	1000 metres	1 km	= 1000 m

IMPERIAL CONVERSIONS				
1 foot	=	12 inches	1 ft	= 12 in
1 yard	=	3 feet	1 yd	= 3 ft
1 chain	=	22 yards	1 ch	= 22 yd
1 furlong	=	220 yards	1 fur	= 220 yd
1 mile	=	1760 yards	1 mi	= 1760 yd

METRIC -> IMPERIAL CONVERSIONS				
1 millimetre	=	0.03937 inches	1 mm	= 0.03937 in
1 centimetre	=	0.3937 inches	1 cm	= 0.39370 in
1 metre	=	39.37008 inches	1 m	= 39.37008 in
1 metre	=	3.28084 feet	1 m	= 3.28084 ft
1 metre	=	1.09361 yards	1 m	= 1.09361 yd



1 foot	=	12 inches	1 ft	= 12 in
1 yard	=	3 feet	1 yd	= 3 ft
1 chain	=	22 yards	1 ch	= 22 yd
1 furlong	=	220 yards	1 fur	= 220 yd
1 mile	=	1760 yards	1 mi	= 1760 yd

METRIC -> IMPERIAL CONVERSIONS				
1 millimetre	=	0.03937 inches	1 mm	= 0.03937 in
1 centimetre	=	0.3937 inches	1 cm	= 0.39370 in
1 metre	=	39.37008 inches	1 m	= 39.37008 in
1 metre	=	3.28084 feet	1 m	= 3.28084 ft
1 metre	=	1.09361 yards	1 m	= 1.09361 yd
1 kilometre	=	1093.6133 yards	1 km	= 1093.6133 yd
1 kilometre	=	0.62137 miles	1 km	= 0.62137 mi

IMPERIAL -> METRIC CONVERSIONS				
1 inch	=	2.54 centimetres	1 in	= 2.54 cm
1 foot	=	30.48 centimetres	1 ft	= 30.48 cm
1 yard	=	91.44 centimetres	1 yd	= 91.44 cm
1 yard	=	0.9144 metres	1 yd	= 0.9144 m
1 mile	=	1609.344 metres	1 mi	= 1609.344 m
1 mile	=	1.609344 kilometres	1 mi	= 1.609344 km

- **Part 3 – Number Generation using Looping Statements**
  - **HTML/PHP**

index.php

3t2526 > module1 > tfa2 > act3 > index.php > html

```php
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <title>Formative 2 - Act 3 - Looping Statements</title>
</head>
<body>
    <div class="output">
        <?php
        for ($i = 0; $i <= 9; $i++) {
            for ($j = 0; $j <= 9; $j++) {
                echo $i . $j . ", ";
            }
        }
        ?>
    </div>
</body>
</html>
```

- **CSS**

index.php | style.css

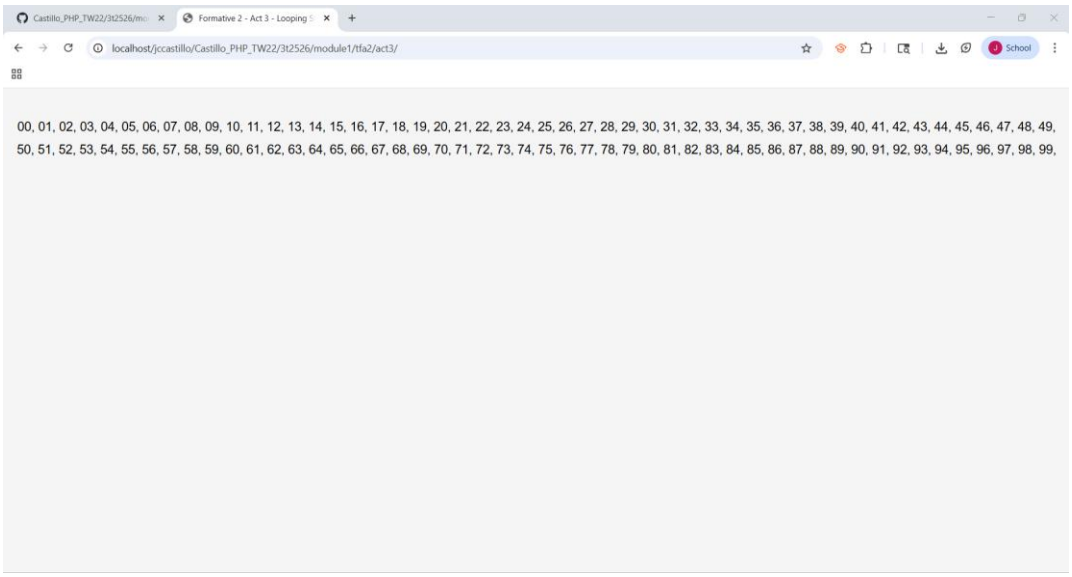
3t2526 > module1 > tfa2 > act3 > # style.css > body

```css
body {
    font-family: Arial, sans-serif;
    background: #f5f5f5;
    text-align: center;
}

.output {
    font-size: 18px;
    line-height: 1.8;
    margin-top: 40px;
}

```

- **Website Output**



VII. QUESTION AND ANSWER

1. What are the different Arithmetic operators used in the lab activity? describe each
   The arithmetic operators used in the lab activity are addition for summing basic values, subtraction for finding differences or decreased values, multiplication for repeated addition, division for quotient calculation, and modulus for finding the division remainder.

2. Can you apply formula in webpages? Give an example
   Yes, they could which an example of it is basic conversion of meters to centimeters by multiplying the input value by 100 upon its specific syntax.
3. What are the different conditional statements? Describe each
   Conditional statements are if for a single condition, if else for two, if-elseif-else for multiple conditions and switch for selection of one of many blocks to be executed.

4. Can you create a condition inside of a condition?
   Yes, which they are called nested conditional statements where for example, an if statement can be placed inside another if or else block.

5. Do you think these conditional statements is important in the program? Then why?
   Yes, they are important for they allow programs to make decisions and respond differently based on input or conditional outputs.

6. What are the different looping statements in PHP? Describe each
   The different looping statements in PHP includes while which loops through a block of code as long as the specified condition is true, do while loops do once then repeats it as long as it is true, for loops do a specific number of times assigned and for each do for each element in an array.

7. What is the importance of looping?
   It reduces code repetition and efficiently handles repetitive tasks such as displaying data or generating sequences.

VIII. REFERENCES

1. https://www.w3schools.com/css/
2. https://www.w3schools.com/html/
3. https://www.w3schools.com/php/php_variables.asp
4. https://www.w3resource.com/php/operators/arithmetic-operators.php
5. https://www.tutorialspoint.com/php/php_arithmatic_operators_examples.htm
6. https://www.math10.com/en/algebra/convenrsion-factors-length-area-volume-mass-speed-energy-power-force.html
7. https://www.w3schools.com/php/php_if_else.asp
8. https://www.w3schools.com/php/php_switch.asp
9. https://www.foxinfotech.in/2019/01/php-form-example-student-grading-system.html
10. https://www.w3schools.com/php/php_looping.asp
11. https://www.w3schools.com/php/php_looping_while.asp
12. https://www.w3schools.com/php/php_looping_do_while.asp
13. https://www.w3schools.com/php/php_looping_for.asp
14. https://www.w3schools.com/php/php_looping_foreach.asp
15. https://www.w3schools.com/php/php_looping_break.asp

Note: The following rubrics/metrics will be used to grade students' output.

Program (100 pts.)	**(Excellent)**	**(Good)**	**(Fair)**	**(Poor)**
Program execution (20pts)	Program executes correctly with no syntax or runtime errors **(18-20pts)**	Program executes with less than 3 errors **(15-17pts)**	Program executes with more than 3 errors **(12-14pts)**	Program does not execute **(10-11pts)**
Correct output (20pts)	Program displays correct output with no errors **(18-20pts)**	Output has minor errors **(15-17pts)**	Output has multiple errors **(12-14pts)**	Output is incorrect **(10-11pts)**
Design of output (10pts)	Program displays more than expected **(10pts)**	Program displays minimally expected output **(8-9pts)**	Program does not display the required output **(6-7pts)**	Output is poorly designed **(5pts)**
Design of logic (20pts)	Program is logically well designed **(18-20pts)**	Program has slight logic errors that do no significantly affect the results **(15-17pts)**	Program has significant logic errors **(3-5pts)**	Program is incorrect **(10-11pts)**
Standards (20pts)	Program code is stylistically well designed **(18-20pts)**	Few inappropriate design choices (i.e. poor variable names, improper indentation) **(15-17pts)**	Several inappropriate design choices (i.e. poor variable names, improper indentation) **(12-14pts)**	Program is poorly written **(10-11pts)**
Delivery (10pts)	The program was delivered on time. **(10pts)**	The program was delivered a day after the deadline. **(8-9pts)**	The program was delivered two days after the deadline. **(6-7pts)**	The program was delivered more than two days after the deadline. **(5pts)**